> aula_6.6 --run

Validação cruzada implementada na mão

K-Fold e Stratified K-Fold escritos como geradores de índices, até chegar ao seu próprio cross_val_score.

DURAÇÃO
2 horas

FERRAMENTAS
Python · NumPy

ENTREGA
meu_cross_val_score()

Objetivos da aula

Ao final, você vai conseguir:

01 Explicar por que uma única divisão treino/teste dá uma estimativa instável.

02 Construir os índices de treino e validação do K-Fold com NumPy.

03 Implementar um Stratified K-Fold que preserva a proporção das classes.

04 Escrever o seu próprio cross_val_score, com um modelo novo por rodada.

05 Resumir os scores com média e desvio-padrão, sem vazamento de dados.

120 MINUTOS

10	Objetivos e recapitulação
20	Divisão única e K-Fold
25	Índices e estratificação
20	Scores, média, desvio e vazamento
35	Exercício guiado
10	Quiz, erros comuns e desafio

O que já sabemos fazer

X, y

X: uma linha por amostra, uma coluna por característica. y: um rótulo por amostra.

→

split

Uma parte para treino, outra guardada para teste.

→

fit

O modelo ajusta seus parâmetros com o treino.

→

predict

Previsões para amostras que o modelo não viu.

→

métrica

Matriz de confusão → accuracy, precision, recall, F1 (aula 6.5).

A PERGUNTA DE HOJE

O número no fim mede o modelo, ou a divisão que sorteamos?

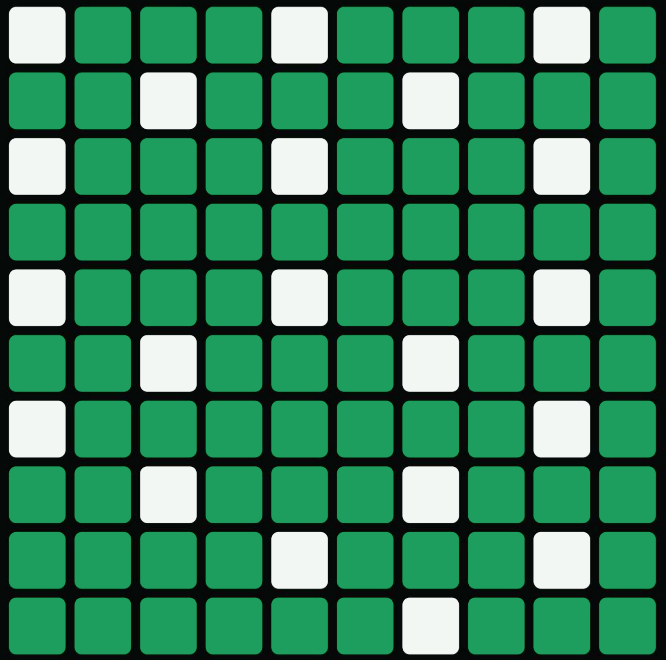
PARTE

01

Por que validar mais de uma vez

- uma divisão treino/teste
- a ideia da validação cruzada
- K, fold e rodada
- três divisões, três resultados
- cinco rodadas, cinco modelos
- K-Fold com 12 amostras

Uma divisão treino/teste



Treino · 80 amostras

O modelo ajusta os parâmetros aqui (fit).

Teste · 20 amostras

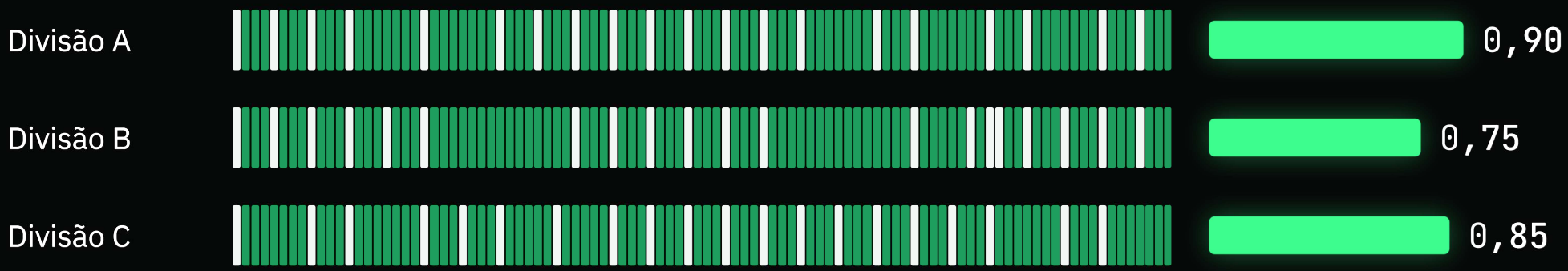
Nunca vistas no treino. Medimos a métrica aqui (predict).

$$accuracy = \frac{\text{acertos}}{20}$$

Cada amostra de teste vale 1/20 = 0,05 da accuracy.

Três divisões, três resultados

Mesmo modelo, mesmos dados. Só mudou quais 20 amostras ficaram no teste.



treino teste accuracy

Valores ilustrativos, não são de uma execução real. 0,90 – 0,75 = 15 pontos = 3 amostras de teste.

A ideia da validação cruzada

1 Separar uma parte para validação

2 Treinar um modelo novo com as outras quatro

3 Avaliar nas amostras reservadas

4 Registrar o resultado

5 Repetir com a próxima parte

NO FINAL cada parte foi validação uma vez 5 treinamentos 5 scores

Cinco rodadas, cinco modelos novos

Cada rodada começa do zero. Continuar treinando o modelo anterior significa avaliá-lo em amostras que ele já viu.

Scores ilustrativos

K, fold e rodada

K

Número de partes. Também é o número de rodadas e de treinamentos.

fold

Uma das K partes: um grupo de amostras.

rodada

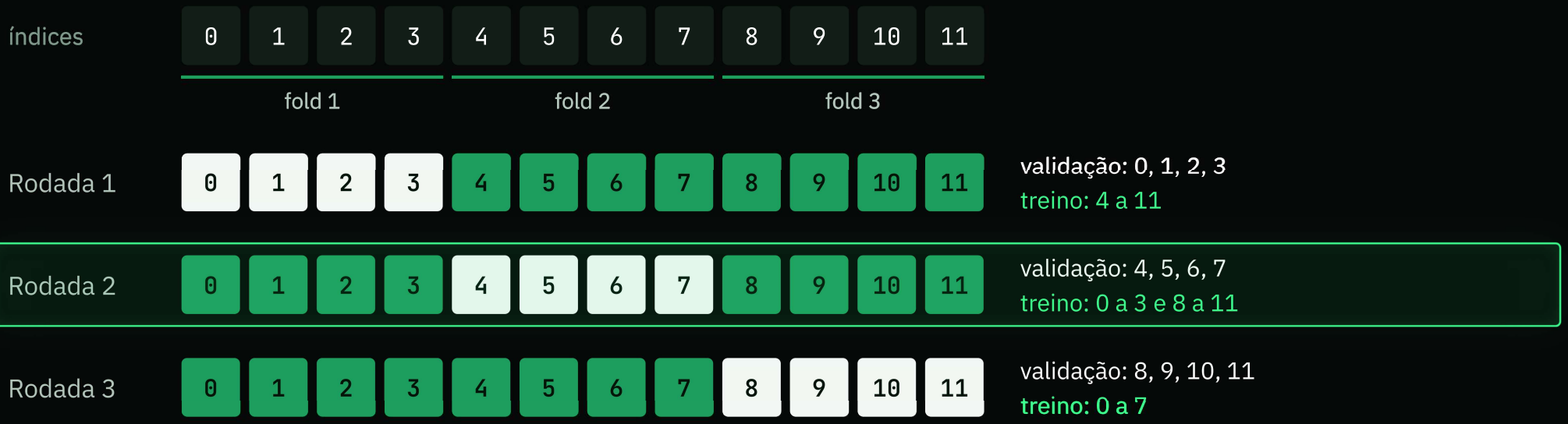
Treinar com K – 1 folds e validar no fold restante.

Com n = 100 amostras

K	Rodadas	Treino por rodada	Validação por rodada
3	3	≈ 67	≈ 33
5	5	80	20
10	10	90	10
n	100	99	1

K maior: mais dados no treino, mais treinamentos, folds de validação menores. 5 e 10 são as escolhas usuais.

K-Fold com 12 amostras e K = 3



Os folds de validação não se sobrepõem.

Cada índice é validação exatamente uma vez.

A amostra 5 treina nas rodadas 1 e 3 e valida na rodada 2.

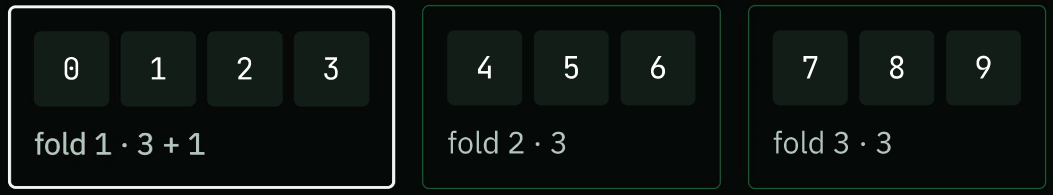
Quando n não é múltiplo de K

`n = 10, K = 3`

`n // K = 3` divisão inteira: tamanho base de cada fold

`n % K = 1` resto: amostras que sobraram

Os primeiros resto folds recebem uma amostra extra.



`tamanhos = [4, 3, 3] soma = 10`

Folds diferem em no máximo uma amostra. É a mesma regra do KFold do scikit-learn.

Quiz rápido · parte 1

Q1 O que significa K em K-Fold?

Q2 O que é um fold?

Q3 Com K = 5, quantas rodadas de treino acontecem?

Q4 Por que não reaproveitamos o modelo treinado na rodada anterior?

PARTE

02

Índices, geradores e K-Fold em NumPy

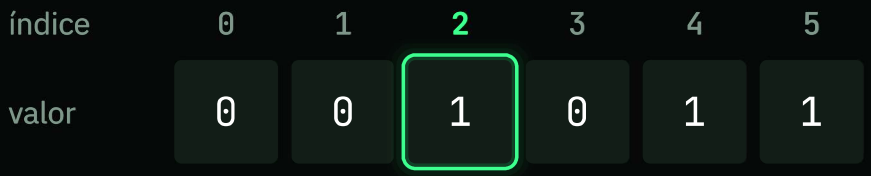
- índice e valor
- embaralhar índices
- fatiamento e concatenação
- selecionar linhas com índices
- funções geradoras e yield
- meu_kfold, linha a linha

Índice e valor

```
indices = np.arange(12)
# [ 0  1  2  3  4  5  6  7  8  9 10 11]

y = np.array([0, 0, 1, 0, 1, 1])
y[2]     # → 1
```

`np.arange(n)`	inteiros de 0 até n – 1; o n fica de fora
`índice`	posição: quantos passos desde o início
`valor`	o que está guardado naquela posição



y[2] lê a posição 2 e devolve o valor 1. O K-Fold escolhe posições; nunca altera valores.

Selecionando linhas com arrays de índices

idx	X[:, 0]	X[:, 1]	y
0	5.1	3.5	0
1	4.9	3.0	0
2	6.2	2.9	1
3	5.9	3.0	1
4	6.7	3.1	1
5	5.0	3.4	0

```
train_idx = np.array([0, 2, 3, 5])
val_idx   = np.array([1, 4])

X_treino = X[train_idx]   # linhas 0, 2, 3, 5
X_val    = X[val_idx]     # [[4.9 3.0] [6.7 3.1]]
y_treino = y[train_idx]   # [0 1 1 0]
y_val    = y[val_idx]     # [0 1]
```

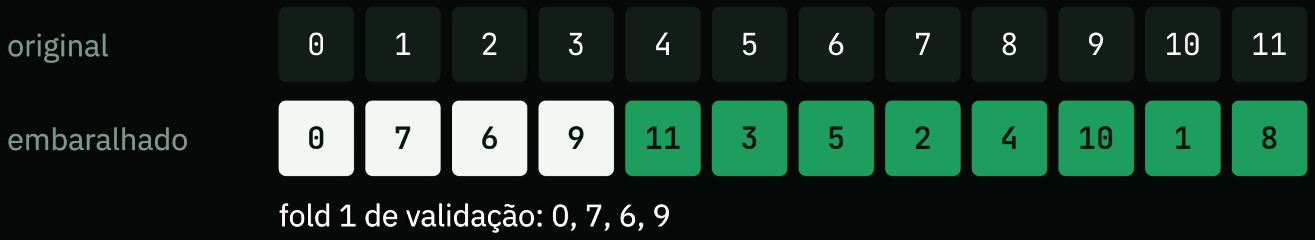
O mesmo array de índices em X e em y mantém a linha `i` ligada ao rótulo `i`.

Embaralhar os índices, não os dados

```
rng = np.random.default_rng(42)   # 42 = semente
indices = rng.permutation(12)
# [ 0  7  6  9 11  3  5  2  4 10  1  8]
```

`shuffle=True` embaralha a ordem antes de cortar os folds

`random_state` fixa a semente: mesma permutação em toda execução



Incorreto

Embaralhar X e y separadamente: cada linha ganha o rótulo de outra amostra.

Correto

Embaralhar os índices uma vez e aplicar os mesmos índices em X e em y.

Funções geradoras e yield

`return` · entrega tudo e encerra

```
def lista_ate(n):
    resultado = []
    for i in range(n):
        resultado.append(i * 10)
    return resultado
```

`yield` · entrega um valor, pausa e continua

```
def gera_ate(n):
    for i in range(n):
        yield i * 10

# sem lista: cada valor nasce
# só quando o for pede
```

```
for valor in gera_ate(3):
    print(valor)
# 0
# 10
# 20
```

Cada volta do `for` pede o próximo `yield`.

No K-Fold, cada `yield` entrega um par `(train_idx, val_idx)`.

Um gerador se esgota: percorrido uma vez, fica vazio.

Fatiamento e concatenação

`indices = np.arange(10)`. Regra da fatia `[inicio:fim]`: o início entra, o fim fica de fora.

`indices[4:7]`

`[4 5 6]`
da posição 4 até antes da 7

`indices[:4]`

`[0 1 2 3]`
do início até antes da 4

`indices[7:]`

`[7 8 9]`
da posição 7 até o fim

`np.concatenate([indices[:4], indices[7:]])`

`[0 1 2 3 7 8 9]`
junta os pedaços em um array

`np.full(3, 10 // 3)` → `[3 3 3]`
3 posições, todas iguais ao tamanho base

`tamanhos[:1] += 1` → `[4 3 3]`
+1 nas primeiras 10 % 3 = 1 posições

meu_kfold · preparação

```python
def meu_kfold(n_amostras, k=5, shuffle=False, random_state=None):
    # 1 validar K
    if not 2 <= k <= n_amostras:
        raise ValueError("k precisa estar entre 2 e n_amostras")
    # 2 criar a sequência de índices
    indices = np.arange(n_amostras)
    # 3 embaralhar (opcional)
    if shuffle:
        rng = np.random.default_rng(random_state)
        indices = rng.permutation(indices)
    # 4 tamanho de cada fold
    tamanhos = np.full(k, n_amostras // k)
    tamanhos[: n_amostras % k] += 1
```

Rastreando n_amostras = 10, k = 3

`2 <= 3 <= 10`
válido, segue

`indices`
`[0 1 2 3 4 5 6 7 8 9]`

`np.full(3, 10 // 3)`
`[3 3 3]`

`tamanhos[:1] += 1`
`[4 3 3]`

meu_kfold · laço dos folds

```
    inicio = 0
    # 5 percorrer os folds
    for tamanho in tamanhos:
        fim = inicio + tamanho
        # 6 fold de validação
        val_idx = indices[inicio:fim]
        # 7 treino = todo o resto
        train_idx = np.concatenate(
            [indices[:inicio], indices[fim:]])
        # 8 entregar o par e pausar
        yield train_idx, val_idx
        inicio = fim
```

n_amostras = 10, k = 3, tamanhos = [4 3 3]

rod.	inicio	fim	índices
1	0	4	val [0 1 2 3] trn [4 5 6 7 8 9]
2	4	7	val [4 5 6] trn [0 1 2 3 7 8 9]
3	7	10	val [7 8 9] trn [0 1 2 3 4 5 6]

inicio = fim faz o próximo fold começar onde o anterior terminou. Sem essa linha, o primeiro fold se repete K vezes.

meu_kfold · saída real e verificação

```python
for fold, (train_idx, val_idx) in enumerate(meu_kfold(10, k=3), start=1):
    print(f"Fold {fold} | treino: {train_idx} | val: {val_idx}")
```

```
Fold 1 | treino: [4 5 6 7 8 9] | val: [0 1 2 3]
Fold 2 | treino: [0 1 2 3 7 8 9] | val: [4 5 6]
Fold 3 | treino: [0 1 2 3 4 5 6] | val: [7 8 9]
```

Cada índice é validação uma vez

```python
vals = np.concatenate([v for _, v in meu_kfold(10, k=3)])
np.array_equal(np.sort(vals), np.arange(10))
# True
```

Treino e validação não se cruzam

```python
all(len(np.intersect1d(t, v)) == 0
    for t, v in meu_kfold(10, k=3))
# True
```

PARTE

03

Estratificação

- classes desbalanceadas
- quando estratificar
- meu_stratified_kfold
- K-Fold × Stratified K-Fold
- estratificação passo a passo
- classes por fold

Classes desbalanceadas nos folds

classe 0 · 90 classe 1 · 10

Dataset ordenado por classe, K-Fold com K = 5, sem embaralhar.

fold 1
0 positivos

fold 2
0 positivos

fold 3
0 positivos

fold 4
0 positivos

fold 5
10 positivos

Rodadas 1 a 4

Validação sem nenhum positivo. Recall da classe 1 = 0/0.

Rodada 5

Treino sem nenhum positivo. O modelo nunca viu a classe 1.

Estratificação

Preservar aproximadamente a proporção das classes em cada fold: 2 positivos em cada fold de 20.

K-Fold e Stratified K-Fold lado a lado

y, 12 amostras — 0 1 2 3 4 5 6 7 **8 9 10 11** — 8 da classe 0 · 4 da classe 1 · K = 4

K-Fold

corta posições seguidas, sem olhar y

fold 1	fold 2	fold 3	fold 4
0	3	6	**9**
1	4	7	**10**
2	5	**8**	**11**
3 : 0	3 : 0	2 : 1	0 : 3

Stratified K-Fold

reparte cada classe entre os folds

fold 1	fold 2	fold 3	fold 4
0	2	4	6
1	3	5	7
8	**9**	**10**	**11**
2 : 1	2 : 1	2 : 1	2 : 1

Contagem por fold = classe 0 : classe 1. O dataset inteiro tem proporção 8 : 4 = 2 : 1.

Quando estratificar

K-Fold

Regressão: y contínuo, sem classes para preservar.

Classes balanceadas e dados embaralhados.

Stratified K-Fold

Classificação, sobretudo com classes desbalanceadas.

Padrão do cross_val_score do sklearn para classificadores com cv inteiro.

Limite 1 · menos exemplos que folds

3 positivos e K = 5: pelo menos dois folds ficam sem positivos. O sklearn avisa; a nossa versão recusa.

Limite 2 · não corrige o desbalanceamento

Cada fold herda a proporção do dataset. Prever sempre 0 ainda dá accuracy 0,90 com 90/10.

Estratificação passo a passo

`y = [0 0 0 0 0 0 0 0 1 1 1 1], K = 4`

01	`np.unique(y, return_counts=True)`	`classes [0 1] · contagens [8 4]` quais classes existem e quantas amostras cada uma tem

02	`np.flatnonzero(y == 1)`	`y == 1 → [F F F F F F F F T T T T]`
		`[ 8 9 10 11]`
		seleciona pelo rótulo, devolve posições

03	`np.array_split(idx_c, 4)`	`classe 0 → [0 1] [2 3] [4 5] [6 7]` `classe 1 → [8] [9] [10] [11]`
04	fold f = parte f de cada classe	`[0 1 8] [2 3 9] [4 5 10] [6 7 11]` cada fold: 2 da classe 0 e 1 da classe 1
05	`np.setdiff1d(todos, val_idx)`	`[ 2 3 4 5 6 7 9 10 11]` treino do fold 1: todos os índices fora de [0 1 8]

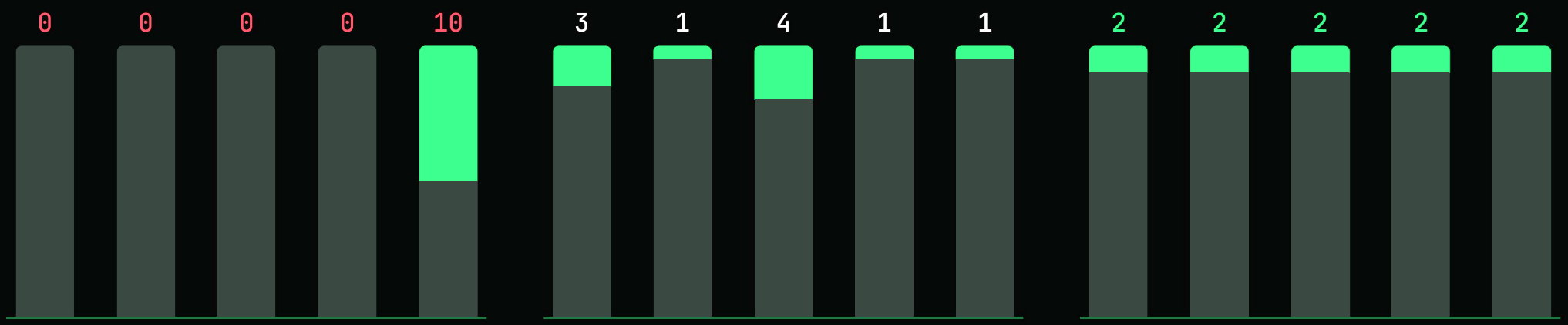
meu_stratified_kfold

```python
def meu_stratified_kfold(y, k=5, shuffle=False, random_state=None):
    y = np.asarray(y)
    classes, contagens = np.unique(y, return_counts=True)   # 1
    if k < 2 or contagens.min() < k:
        raise ValueError("cada classe precisa ter pelo menos k amostras")
    rng = np.random.default_rng(random_state)
    partes_por_fold = [[] for _ in range(k)]                # 2
    for c in classes:
        idx_c = np.flatnonzero(y == c)                      # 3
        if shuffle:
            idx_c = rng.permutation(idx_c)
        for f, parte in enumerate(np.array_split(idx_c, k)): # 4
            partes_por_fold[f].append(parte)
    todos = np.arange(len(y))
    for partes in partes_por_fold:
        val_idx = np.sort(np.concatenate(partes))           # 5
        train_idx = np.setdiff1d(todos, val_idx)            # 6
        yield train_idx, val_idx                            # 7
```

1. classes e contagens; recusa se alguma classe tem menos de k
2. uma lista vazia por fold
3. posições da classe c, embaralhadas dentro da classe
4. parte f da classe vai para o fold f
5. juntar as partes: validação
6. complemento: treino
7. entregar o par

Distribuição das classes por fold

Saída real · 100 amostras (90 / 10), K = 5. Número sobre a barra = positivos no fold de validação.



meu_kfold
shuffle=False

meu_kfold
shuffle=True, random_state=42

meu_stratified_kfold
shuffle=True, random_state=42

```
np.bincount(y[val_idx], minlength=2) # [negativos, positivos] de cada fold
```

Quiz rápido · parte 2

Q1 Qual é o papel dos índices no K-Fold?

Q2 O que acontece quando um fold de validação tem poucas amostras da classe positiva?

Q3 Qual é a diferença entre K-Fold e Stratified K-Fold?

PARTE

04

Scores, média, desvio e cross_val_score

- uma métrica por fold
- desvio-padrão
- fábrica de modelos
- vazamento de dados
- média dos scores
- dispersão entre modelos
- meu_cross_val_score
- teste final e limites

Uma métrica por fold

EM CADA RODADA

1. Criar um modelo novo
2. Treinar com `X[train_idx], y[train_idx]`
3. Prever em `X[val_idx]`
4. Comparar com `y[val_idx]`: accuracy
5. Guardar em `scores`

```
accuracy = np.mean(y_val == y_pred) # aula 6.5
```

Fold	Accuracy	0 — 1
1	0,80	
2	0,85	
3	0,90	
4	0,75	
5	0,85	

Valores ilustrativos

Média dos scores

FÓRMULA · MÉDIA

$$\bar{s} = \frac{1}{K} \sum_{k=1}^{K} s_k$$

$\bar{s}$	média dos scores	s_k	score do fold k
K	número de folds (aqui, 5)	Σ	soma de k = 1 até K

CÁLCULO

```
  0,80
+ 0,85
+ 0,90
+ 0,75
+ 0,85
------
  4,15
÷ 5
= 0,83
```

`np.mean(scores)` → `0.83`

Desvio-padrão

FÓRMULA · DESVIO-PADRÃO (POPULACIONAL)

$$\sigma = \sqrt{\frac{1}{K}\sum_{k=1}^{K}(s_k - \bar{s})^2}$$

01 desvio de cada fold: $s_k - \bar{s}$

02 ao quadrado: sinais não se cancelam

03 média dos quadrados (÷ K): variância

04 raiz: volta à unidade dos scores

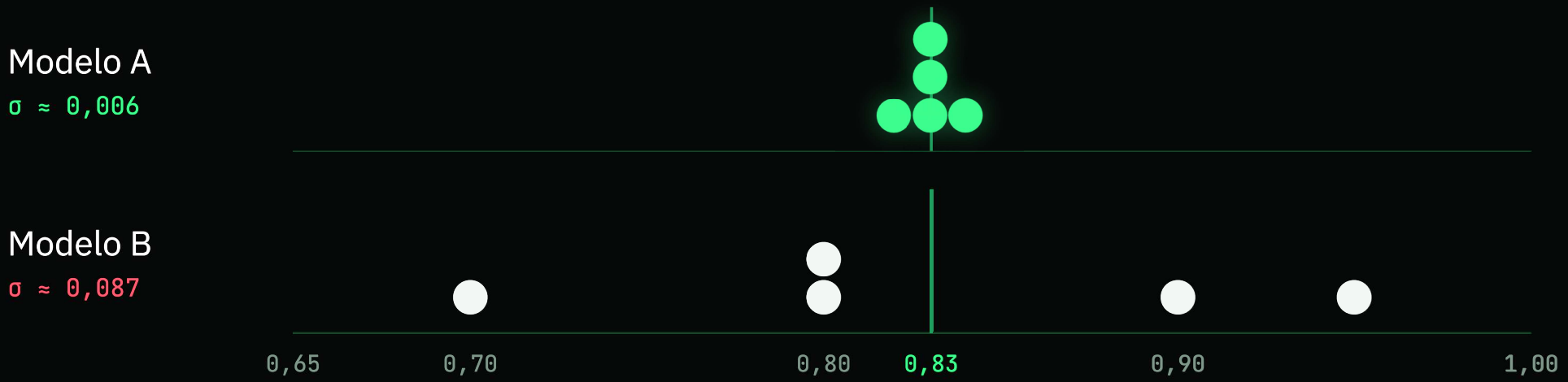
s_k	$s_k - \bar{s}$	$(\ldots)^2$
0,80	-0,03	0,0009
0,85	+0,02	0,0004
0,90	+0,07	0,0049
0,75	-0,08	0,0064
0,85	+0,02	0,0004

soma = 0,0130
÷ 5 = 0,0026
√ ≈ 0,051

`np.std(scores)` → `0.051`

np.std usa ddof=0 (÷ K). Com ddof=1 (÷ K – 1): ≈ 0,057.

Mesma média, dispersões diferentes



Ambos com média 0,83. O desempenho de B depende muito de quais amostras caem na validação.

σ entre folds descreve a variação observada. Não é intervalo de confiança: os folds compartilham dados de treino.

Fluxo do meu_cross_val_score

entradas `modelo_factory` `X` `y` `folds` `metrica`

1	2	3	4	5	6
Próximo par	**Separar**	**Modelo novo**	**Treinar**	**Prever**	**Avaliar**

`for train_idx, val_idx in folds`	`X[train_idx]`
	`y[train_idx]`
	`X[val_idx]`
	`y[val_idx]`
		`modelo = modelo_factory()`	`modelo.fit( X_treino, y_treino)`	`y_pred = modelo.predict( X_val)`	`scores.append( metrica(y_val, y_pred))`

↺ repete enquanto o gerador entregar pares

saída `return np.array(scores)` → um score por fold; média e desvio ficam com quem chamou

Uma fábrica de modelos

Incorreto · um objeto para todas as rodadas

```
modelo = CentroideMaisProximo()   # uma vez
for train_idx, val_idx in folds:
    modelo.fit(X[train_idx], y[train_idx])
```

Correto · um objeto novo por rodada

```
for train_idx, val_idx in folds:
    modelo = modelo_factory()      # a cada rodada
    modelo.fit(X[train_idx], y[train_idx])
```

Fábrica: função sem argumentos que devolve um modelo novo, ainda sem treino.

```
fabrica = CentroideMaisProximo     # a classe
fabrica = lambda: RegressaoLogistica(taxa=0.1)
m1 = fabrica()
m2 = fabrica()
m1 is m2                           # False
```

Sem parênteses: passamos a fábrica.

Com parênteses: criamos um modelo.

meu_cross_val_score

```python
def meu_cross_val_score(modelo_factory, X, y, folds, metrica):
    scores = []
    for train_idx, val_idx in folds:
        X_treino, y_treino = X[train_idx], y[train_idx]
        X_val, y_val = X[val_idx], y[val_idx]
        modelo = modelo_factory()        # novo a cada rodada
        modelo.fit(X_treino, y_treino)
        y_pred = modelo.predict(X_val)
        scores.append(metrica(y_val, y_pred))
    return np.array(scores)
```

```python
scores = meu_cross_val_score(CentroideMaisProximo, X, y,
    meu_stratified_kfold(y, k=5, shuffle=True, random_state=42), acuracia)
# [0.9  0.9  0.85 0.85 0.95]   0.890 ± 0.037
```

modelo_factory
cria um modelo novo a cada chamada: a classe, ou lambda: Modelo(param=...)

X, y
características e rótulos, na mesma ordem

folds
gerador de pares (train_idx, val_idx)

metrica
função (y_true, y_pred) → número

Vazamento de dados na padronização

Incorreto · estatísticas de todas as amostras

```
media = X.mean(axis=0)   # usa as 100
desvio = X.std(axis=0)
X_pad = (X - media) / desvio

for train_idx, val_idx in folds:
    ...   # a validação já influenciou
          # media e desvio
```

Correto · estatísticas só do treino do fold

```
for train_idx, val_idx in folds:
    media = X[train_idx].mean(axis=0)
    desvio = X[train_idx].std(axis=0)
    X_treino = (X[train_idx] - media) / desvio
    X_val = (X[val_idx] - media) / desvio
    ...   # fit em X_treino, avalia em X_val
```

A regra vale para tudo que aprende com os dados: imputação, seleção de features, codificação pelo alvo, PCA.
Se tem `fit`, fica dentro do fold.

Treino, validação e teste final

Desenvolvimento · 80% · validação cruzada (uma rodada)

Teste final · 20%

Treino dentro do fold

Ajusta os parâmetros do modelo e as estatísticas do pré-processamento.

Validação dentro do fold

Estima o desempenho. Compara modelos e escolhe hiperparâmetros.

Teste final

Aberto uma vez, no fim, com o modelo já escolhido. Não participa da escolha.

Quando K-Fold não é suficiente

Séries temporais

Embaralhar usa o futuro para prever o passado. Validar sempre depois do treino.

`TimeSeriesSplit`

Amostras agrupadas

Várias linhas da mesma pessoa ou entidade. O grupo inteiro fica de um lado só.

`GroupKFold`

Observações dependentes

Vizinhos no tempo ou no espaço se parecem. A validação fica fácil demais.

`folds em blocos contíguos`

Classes muito raras

Poucos positivos por fold: scores instáveis. Repetir com sementes diferentes.

`RepeatedStratifiedKFold`

Quiz rápido · parte 3

Q1 Por que calculamos a média dos scores?

Q2 O que o desvio-padrão entre folds informa?

Q3 O que é vazamento de dados, e por que o pré-processamento fica dentro do fold?

Q4 O teste final deve participar da escolha dos hiperparâmetros?

PARTE

05

Prática no notebook

› exercício guiado em 12 etapas
› erros comuns
› comparação com o scikit-learn
› desafio de casa

Exercício guiado · roteiro

6.6_exercicio_guiado.ipynb · 35 min

Dados — 5 min

1 · criar o dataset

2 · identificar X e y

3 · distribuição das classes

Folds — 12 min

4 · índices do K-Fold

5 · visualizar os folds

6 · Stratified K-Fold

7 · classes por fold

Avaliação — 12 min

8 · modelo novo por rodada

9 · accuracy por fold

10 · média e desvio

11 · meu_cross_val_score

Comparação — 6 min

12 · conferir com o scikit-learn

Até a etapa 12: sem `KFold`, `StratifiedKFold` ou `cross_val_score`.

Etapas 1 a 3 · dados e classes

```python
import numpy as np
rng = np.random.default_rng(0)
X0 = rng.normal(loc=[0, 0], scale=1.0, size=(90, 2))   # classe 0
X1 = rng.normal(loc=[2, 2], scale=1.0, size=(10, 2))   # classe 1

X = np.vstack([X0, X1])
y = np.concatenate([np.zeros(90, dtype=int), np.ones(10, dtype=int)])

print(X.shape, y.shape)
print(np.unique(y, return_counts=True))
```

```
(100, 2) (100,)
(array([0, 1]), array([90, 10]))
```

X · 100 × 2

uma linha por amostra, duas características

y · 100

90 zeros e 10 uns: 10% de positivos

Ordenado por classe

as 90 negativas vêm primeiro. Guarde isso para a etapa 4.

Etapas 4 a 7 · folds e distribuição

```python
def contar_classes(gerador):
    for fold, (train_idx, val_idx) in enumerate(gerador, start=1):
        print(fold, np.bincount(y[val_idx], minlength=2))

contar_classes(meu_kfold(100, k=5))
contar_classes(meu_kfold(100, k=5, shuffle=True, random_state=42))
contar_classes(meu_stratified_kfold(y, k=5,
                                    shuffle=True, random_state=42))
```

Saída real · [classe 0, classe 1] na validação

fold	K-Fold	shuffle	estratif.
1	[20 0]	[17 3]	[18 2]
2	[20 0]	[19 1]	[18 2]
3	[20 0]	[16 4]	[18 2]
4	[20 0]	[19 1]	[18 2]
5	[10 10]	[19 1]	[18 2]

Etapa 5 (gráfico dos folds) está no notebook.

Etapa 8 · modelo de referência

```python
class CentroideMaisProximo:
    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.centroides_ = np.array(
            [X[y == c].mean(axis=0) for c in self.classes_])
        return self

    def predict(self, X):
        dist = np.stack([np.linalg.norm(X - c, axis=1)
                         for c in self.centroides_], axis=1)
        return self.classes_[np.argmin(dist, axis=1)]

def acuracia(y_true, y_pred):
    return np.mean(y_true == y_pred)
```

`fit`
centroide de cada classe = média das linhas da classe

`predict`
distância a cada centroide; vence o mais próximo (argmin)

`acuracia`
fração de acertos (aula 6.5)

Qualquer objeto com `fit` e `predict` serve, como a RegressaoLogistica da aula 6.4.

Etapas 9 a 11 · scores por fold

```
folds = meu_stratified_kfold(y, k=5, shuffle=True, random_state=42)
scores = meu_cross_val_score(CentroideMaisProximo, X, y, folds, acuracia)
print(scores, f"{scores.mean():.3f} ± {scores.std():.3f}")
```

SAÍDA REAL · MESMO MODELO, TRÊS FORMAS DE DIVIDIR

K-Fold sem shuffle

0.90 0.75 0.90 0.95 0.50

0.800 ± 0.164

K-Fold, shuffle (42)

0.80 0.95 0.70 0.95 0.95

0.870 ± 0.103

Stratified, shuffle (42)

0.90 0.90 0.85 0.85 0.95

0.890 ± 0.037

0,50 no fold 5: o treino daquela rodada não tinha nenhum positivo, e o modelo previu 0 para tudo.

0,89 fica abaixo de 0,90, a accuracy de quem sempre responde 0. Em base desbalanceada, accuracy sozinha engana.

Etapa 12 · comparação com o scikit-learn

```python
from sklearn.model_selection import (
    KFold, StratifiedKFold, cross_val_score)
from sklearn.neighbors import NearestCentroid

# 1 mesmos índices sem shuffle?
all(np.array_equal(v1, v2) for (_, v1), (_, v2)
    in zip(meu_kfold(100, k=5), KFold(5).split(X)))
# True

# 2 nossas divisões no cross_val_score oficial
meus_folds = list(meu_stratified_kfold(
    y, k=5, shuffle=True, random_state=42))
cross_val_score(NearestCentroid(), X, y, cv=meus_folds)
# [0.9  0.9  0.85 0.85 0.95]   idêntico

# 3 divisões do próprio sklearn
cv = StratifiedKFold(5, shuffle=True, random_state=42)
cross_val_score(NearestCentroid(), X, y, cv=cv)
# [1.   0.85 0.85 0.85 0.85]   0.880 ± 0.060
```

1 e 2 · idênticos

mesma regra de tamanhos; mesmas divisões dão os mesmos scores

3 · diferente, sem erro

outro gerador aleatório (RandomState × default_rng) e outra regra de alocação; mesma proporção 18 / 2 por fold

KFold sem shuffle no sklearn

o fold 5 vira nan: NearestCentroid recusa treinar com uma única classe

Erros comuns · índices e folds

Situação	Por que é problema	Consequência	Correção
Ler K = 5 como 5 amostras por fold	K é o número de folds	rodadas e tamanhos errados	tamanho ≈ n // K
`metrica(val_idx, y_pred)`	índice é posição, não rótulo	métrica sem sentido, sem erro de execução	`metrica(y[val_idx], y_pred)`
`modelo.fit(X[val_idx], …)`	papéis de treino e validação trocados	treina com 20, avalia em 80	conferir os tamanhos de cada rodada
`indices[fim - 1:]`	fatias sobrepostas ou com buraco	amostra nos dois conjuntos, ou perdida	intersect1d vazio; união = todos
`rng.shuffle(X); rng.shuffle(y)`	duas permutações diferentes	rótulos desalinhados; modelo aprende ruído	permutar índices uma vez

Erros comuns · modelo e dados

Situação	Por que é problema	Consequência	Correção
`modelo.fit(X, y)` antes do laço	o modelo já viu as validações	scores altos que não medem nada	fit dentro do laço, com X[train_idx]
`modelo = Modelo()` antes do `for`	o estado da rodada anterior pode persistir	rodadas podem ficar dependentes e otimistas	`modelo_factory()` dentro do `for`
`shuffle=True` sem semente	folds mudam a cada execução	resultados não reprodutíveis	fixar random_state; embaralhar só dados independentes
K-Fold comum em base 90 / 10	folds sem a classe minoritária	recall 0/0, scores instáveis	Stratified K-Fold e métricas por classe

Erros comuns · avaliação

Situação	Por que é problema	Consequência	Correção
Padronizar X inteiro antes da validação cruzada	a validação influencia a transformação	score otimista	ajustar com X[train_idx] em cada fold
Escolher hiperparâmetros pelo score no teste	o teste vira parte da seleção	estimativa final otimista	escolher pela média da CV; teste uma vez
“Em produção o modelo terá 0,89”	a média estima, não garante	expectativa irreal	reportar média ± desvio, K, estratégia e semente
`shuffle=True` em vendas diárias	treina com o futuro	desempenho que não existe na prática	ordem cronológica (TimeSeriesSplit)

Construa seu próprio cross_val_score

6.6_desafio.ipynb

Parte 1

K-Fold

Gerador de pares de índices de treino e validação, com shuffle e semente.

Parte 2

Stratified K-Fold

Gerador que preserva aproximadamente a proporção das classes.

Parte 3

Cross-val-score

Modelo novo por rodada, métrica, scores, média e desvio-padrão.

ENTREGAR

- índices dos folds
- distribuição de classes
- scores individuais
- média e desvio-padrão
- gráfico dos scores
- comparação com o scikit-learn

Dataset: load_breast_cancer (569 amostras, 212 / 357). Correção no início da aula 6.7.

Critérios de aceitação

01 Cada amostra é validação uma vez no K-Fold.

02 Treino e validação sem índices em comum na mesma rodada.

03 O Stratified K-Fold preserva aproximadamente as proporções.

04 Cada rodada usa um modelo novo.

05 As métricas são calculadas corretamente.

06 Média e desvio-padrão corretos, com o ddof declarado.

07 Resultados reprodutíveis quando uma semente é informada.

08 Código organizado e comentado.

09 O aluno explica o funcionamento das funções.

10 Comparação com o scikit-learn sob divisões equivalentes.

Código inicial do desafio

```python
def meu_kfold(n_amostras, k=5, shuffle=False, random_state=None):
    # TODO 1: validar k (2 <= k <= n_amostras)
    # TODO 2: criar os índices e embaralhar se shuffle
    # TODO 3: calcular os tamanhos com // e %
    # TODO 4: para cada fold, yield train_idx, val_idx
    raise NotImplementedError("TODO 1 a 4")

def meu_stratified_kfold(y, k=5, shuffle=False, random_state=None):
    # TODO 5: classes e contagens com np.unique
    # TODO 6: repartir os índices de cada classe com np.array_split
    # TODO 7: yield train_idx, val_idx (treino = complemento)
    raise NotImplementedError("TODO 5 a 7")

def meu_cross_val_score(modelo_factory, X, y, folds, metrica):
    # TODO 8: em cada rodada: modelo novo, fit, predict, métrica
    raise NotImplementedError("TODO 8")
```

Células de verificação com **assert** testam os critérios 1, 2, 3, 4, 5 e 7.

Comece sem shuffle. Acrescente o embaralhamento depois.

Leitura

NUMPY E PYTHON

numpy.arange
numpy.org/doc/stable/reference/generated/numpy.arange.html

Generator.permutation
numpy.org/doc/stable/reference/random/generated/...permutation.html

numpy.unique
numpy.org/doc/stable/reference/generated/numpy.unique.html

numpy.array_split
numpy.org/doc/stable/reference/generated/numpy.array_split.html

Python · generator
docs.python.org/3/glossary.html#term-generator

SCIKIT-LEARN

KFold
scikit-learn.org/stable/modules/generated/...KFold.html

StratifiedKFold
scikit-learn.org/stable/modules/generated/...StratifiedKFold.html

cross_val_score
scikit-learn.org/stable/modules/generated/...cross_val_score.html

Guia do usuário · Cross-validation
scikit-learn.org/stable/modules/cross_validation.html

Próxima aula

6.7 · KNN do zero